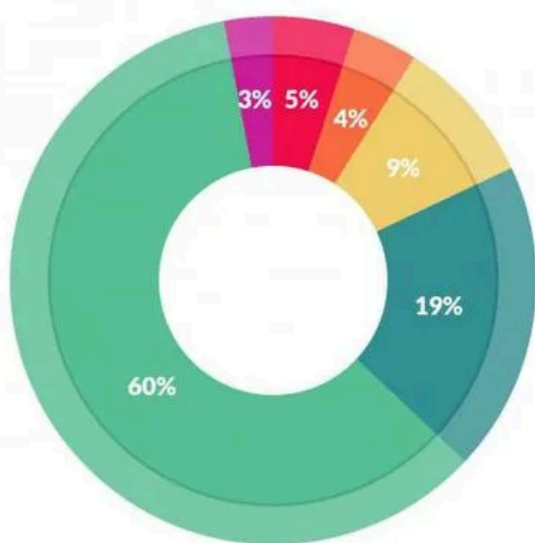


Fasi Pulizia Dati Standard

La **pulizia e la standardizzazione dei dati** costituiscono il pilastro fondamentale per trasformare input grezzi, spesso eterogenei o frammentati, in un patrimonio informativo coerente e ben definito. All'interno di database reali, le discrepanze derivanti da errori di battitura, variazioni ortografiche o formati obsoleti rappresentano ostacoli critici che possono distorcere le analisi successive.

Inoltre è buon uso documentare il processo di pulizia dati e creare una copia su cui lavorare per conservare l'originale.

Es : Passaggi importanti che implicino modifiche, spostamenti e/o rimozione dei dati (nome variabile modificata per renderla più intuitiva) + la motivazione.



What data scientists spend the most time doing

- Building training sets: 3%
- Cleaning and organizing data: 60%
- Collecting data sets: 19%
- Mining data for patterns: 9%
- Refining algorithms: 4%
- Other: 5%

Fonte: articolo forbes

1. Studio base, no modifiche

Ottenuto il dataset andiamo a leggerlo e comprenderlo approfonditamente. In questa fase non si va a modificare niente, bensì andiamo a prendere in considerazione ogni aspetto dei dati: la disposizione, la quantità, le fonti, la tipologia ed eventuali casi particolari che ci

potrebbero dare problemi. Particolare attenzione ai dati mancanti classificare in una maniera specifica per il data set.

Funzioni base:

Controlla struttura e qualità

- `df.info()`
- `df.shape`
- `df.columns`
- `df.dtypes` → Si usa per scoprire il tipo di dati di ciascuna colonna nel DataFrame fornito, ma è anche molto utile per scoprire il tipo di dati di ciascuna colonna nel DataFrame specificato. Questa specifica ti permette di vedere se ci sono dei tipi all'interno della colonna che non sono allineati agli altri permettendo di correggere gli eventuali errori.

```
# return the dtype of each column
```

```
result = df.dtypes
```

```
# Print the result
```

```
print(result)
```

```
A    float64  
B    float64  
C     int64  
D    float64  
dtype: object
```

Statistiche descrittive

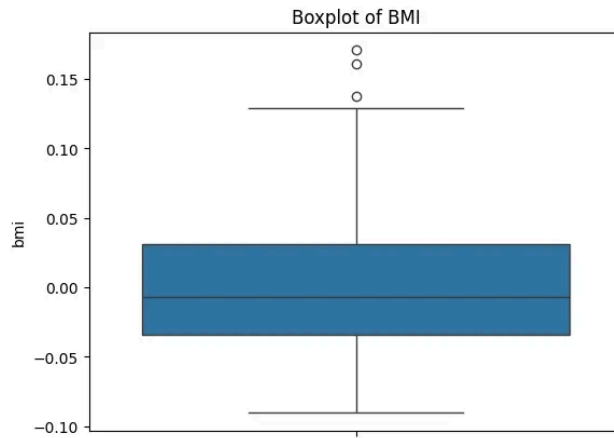
- `df.describe()` → Genera statistiche descrittive delle colonne di un DataFrame, fornendo metriche chiave come media, deviazione standard, percentili e altro ancora. Funziona di default con dati numerici, ma può gestire anche dati categorici, offrendo informazioni come il valore più frequente e il numero di voci univoche.
- `df.describe(include='all')` → Utilizzando il parametro `include='all'` possiamo generare un riepilogo per tutte le colonne del DataFrame, indipendentemente dal tipo di dati. Questo è utile quando si desidera analizzare contemporaneamente dati numerici e categorici.

Valori mancanti

- `df.isnull().sum()`

Outliers + Mappa/grafici per visualizzazione

- `df.boxplot()` → Un diagramma a scatola (box plot) aiuta a visualizzare la distribuzione dei dati utilizzando i quartili. Tutti i punti al di fuori dei baffi del diagramma a scatola sono considerati valori anomali. È un modo semplice per vedere dove si concentra la maggior parte dei valori dei dati e identificare i valori insoliti.



Duplicati

- `df.duplicated().sum()`
- `df[df.duplicated()]`

Analisi valori

- `df["colonna"].value_counts()`
- `df["colonna"].nunique()`

2. Prime modifiche base, conservativo

Dopo la prima analisi andiamo a fare le prime modifiche sul dataset (è consigliabile andare a fare una copia dell'originale per conservare l'originale).

Ecco alcune importanti passaggi di pulizia dei dati iniziali:

Valutazione delle variabili a disposizione

- valutazione dei dati presenti, selezioni delle possibili utili e rinomina secondo uno standard definito dal progetto.

Rimuovi i duplicati

- `df.drop_duplicates()`
- `df.drop_duplicates(inplace = True)` → L'utilizzo di `inplace=True` modifica direttamente il DataFrame originale, risparmiando memoria ed evitando la necessità di assegnare il risultato a una nuova variabile.

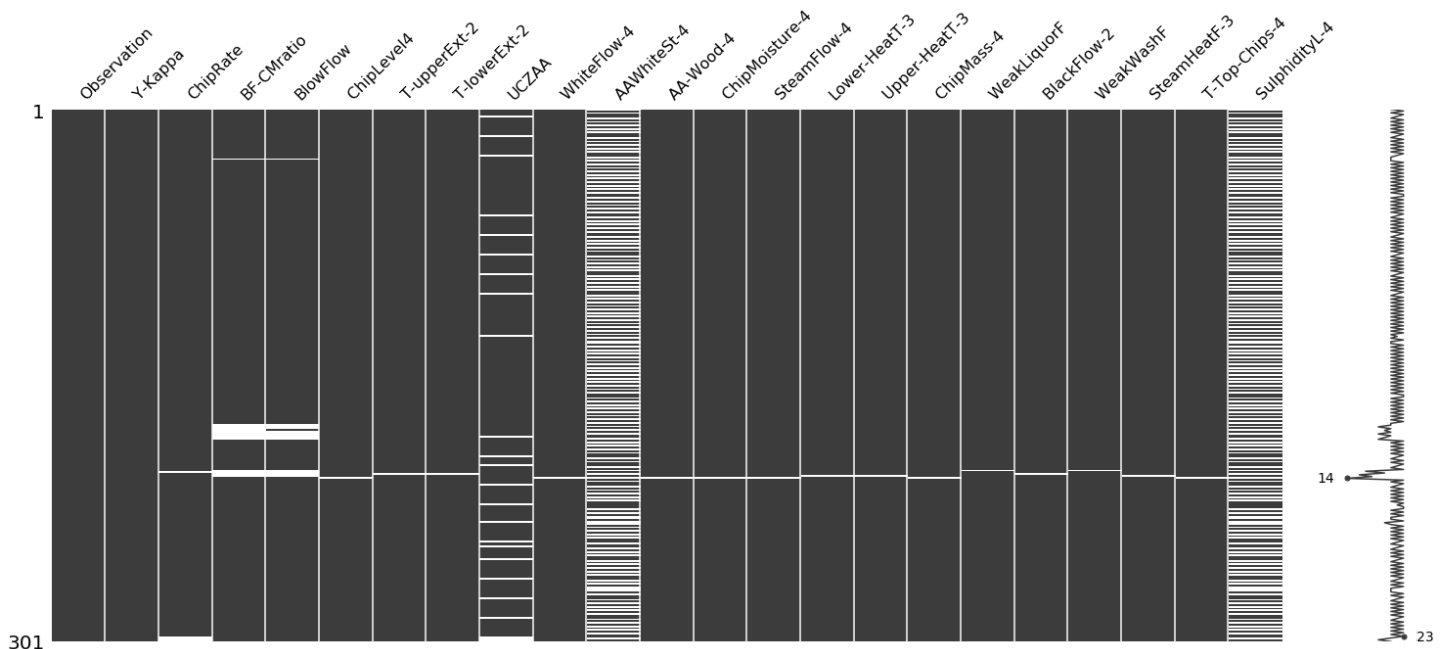
Verifica e gestione dei valori mancanti (aiutarti con un grafico, group by, ragionando per tipo di parametro anche funzione se necessario)

- None = oggetto Python per i valori mancanti negli array di tipo oggetto.
- NaN = oggetto Python per i valori mancanti negli array di tipo float.
- Grafici per visualizzare dati mancanti (HEATED MAP TRA VARIABILI E VARIABLE-SOGGETTO):
 - **Matrix** ⇒

```
msno.matrix(df)

plt.show()
```

Questo comando permette di individuare rapidamente i pattern di dati mancanti nel dataset.

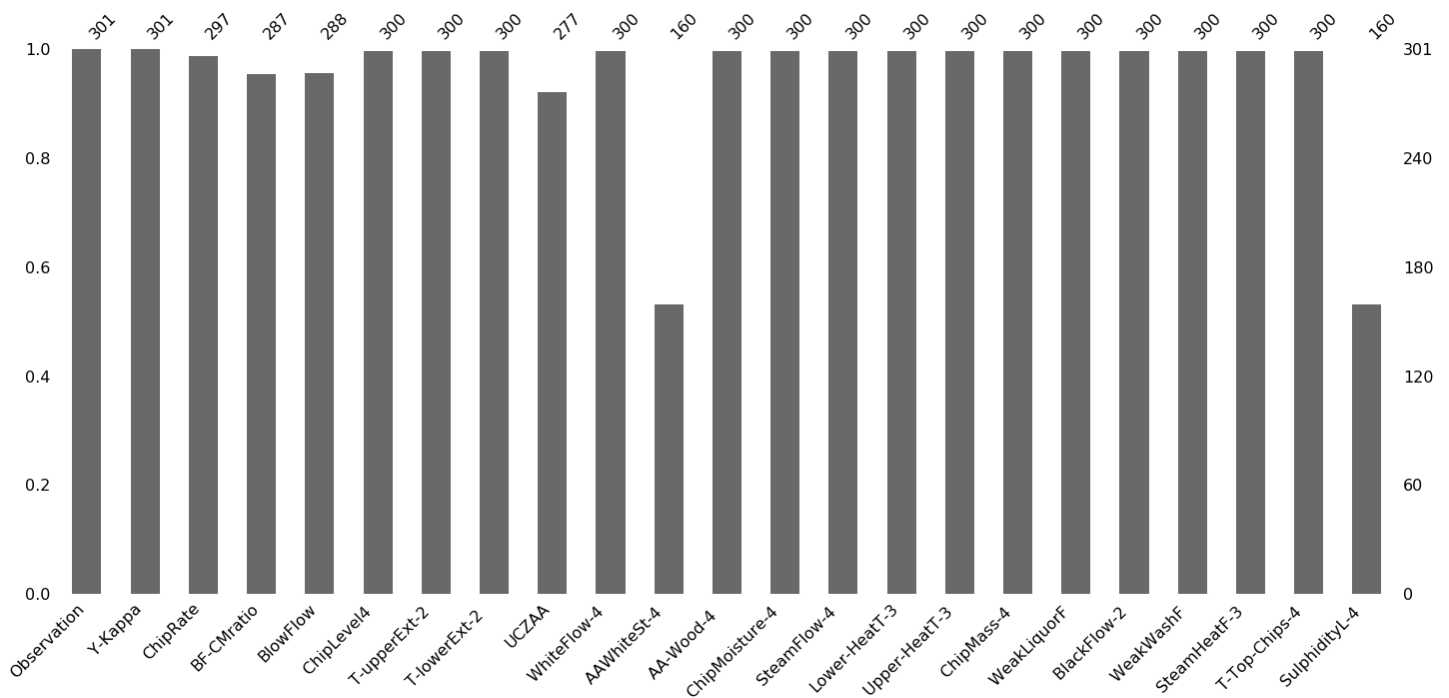


- **Bar Chart** ⇒

```
msno.bar(df)

plt.show()
```

Fornisce un'idea immediata di quanti valori mancanti sono presenti in ogni colonna.



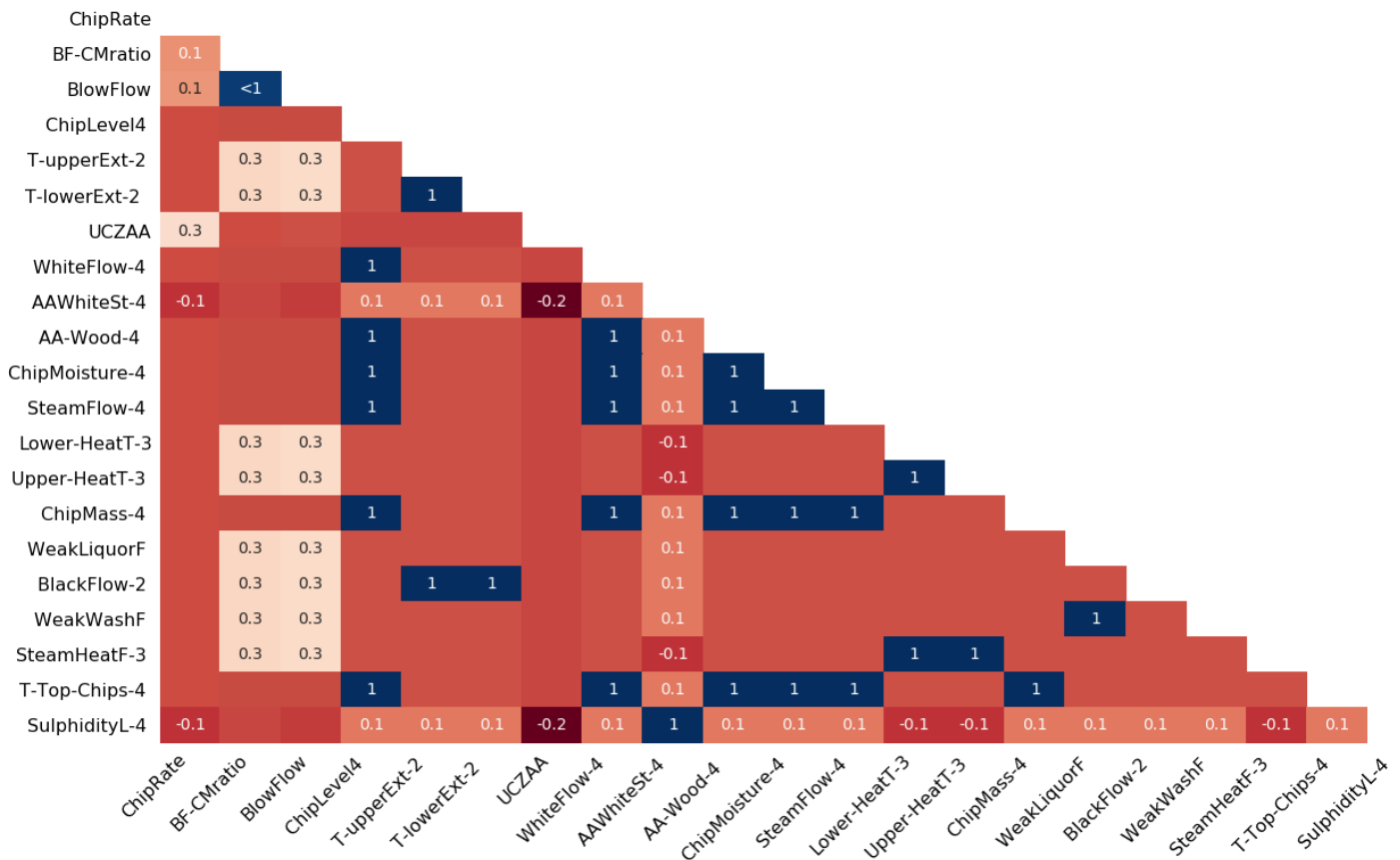
○ **Heatmap** ⇒

```
msno.heatmap(df)

plt.show()
```

Mostra la correlazione della "mancanza" tra ogni coppia di colonne.

- Un valore vicino a **1** indica che se un dato è presente in una colonna, è molto probabile che sia presente anche nell'altra.
- Un valore vicino a **-1** indica che se una variabile appare, è molto probabile che l'altra sia mancante.
- Un valore vicino a **0** indica che non c'è dipendenza tra la mancanza di dati nelle due variabili.



- `Dataframe.isnull()` → restituisce dei dati booleani che semplificano il conteggio/analisi dei dati mancanti
- `groupby()` + `sum()`, `mean()`, `count()` → queste funzioni insieme permettono di farsi un'idea sulla quantità, percentuale degli elementi che si sta analizzando
- funzione per identificare le righe dove una lista definita di colonne ha solo valori nulli.

→ commentare quando e perchè valutare di cancellare i Nan, cosa succede? elimino una riga intera, quindi anche valori validi o tutta la colonna

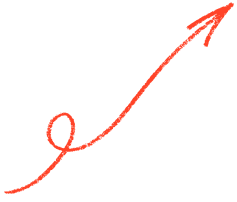
in gener quando è consigliato eliminare colonne e/o righe per valori mancanti?

• Come decidere se eliminare i valori mancanti?

Non esiste una risposta univoca alla domanda se eliminare o meno i valori mancanti. La decisione dipende dalla natura e dall'entità dei dati mancanti nel dataset, dallo scopo e dall'ambito dell'analisi, nonché dalla disponibilità e dalla qualità di strategie alternative per la loro gestione. È opportuno valutare la quantità di valori mancanti, la loro distribuzione e se siano assenti in modo casuale. Inoltre, occorre considerare le domande e le ipotesi di ricerca, i metodi e le tecniche impiegati e la loro sensibilità alla presenza di dati mancanti. Infine, è necessario valutare i compromessi e le ipotesi sottostanti alle procedure di imputazione, sostituzione o esclusione dei valori mancanti, nonché l'affidabilità e la validità di tali approcci. aggiunge domande a step

- `dropna()` → Rimuovi le righe che contengono almeno un valore mancante

- `dropna(how='all')` → eliminare le righe in cui mancano tutti i valori
- `dropna(axis=1)` → rimuovere le colonne che contengono almeno un valore mancante
- `colonne_vuote = df.columns[df.isnull().all()]` → visualizzare le colonne contenenti solo dati mancanti



Individuare e rimuovere i valori Outliers

Chiedere a chiara come inserire i dati → FOCUS su come valuto e studio gli outliers e poi nel caso se decido di eliminarli, come faccio? Diventeranno dei Nan

- **Esclusione:** in alcuni casi, l'esclusione dell'outlier dall'analisi può essere giustificata, soprattutto se si ritiene che esso sia il risultato di un errore di misurazione o registrazione. Tra i metodi di esclusione utilizzati vi è il trimming, cioè l'eliminazione di una percentuale fissa di valori estremi. La decisione di escludere i valori anomali o una percentuale fissa di essi deve essere ben documentata e giustificata per evitare bias.
- **Mantenere:** La gestione degli outlier richiede estrema trasparenza e una profonda analisi del contesto. Consideriamo, ad esempio, uno studio sul rapporto tra peso e altezza: un peso molto elevato potrebbe apparire inizialmente come un errore statistico da eliminare. Tuttavia, se incrociamo il dato con l'altezza, potremmo scoprire che il valore è coerente perché riferito a una persona estremamente alta.
 - In questo caso, mantenere il dato è fondamentale perché:
 - **Rappresenta la variabilità reale:** Eliminarlo significherebbe ignorare casi rari ma esistenti, creando un modello troppo semplificato.
 - **Evita interpretazioni errate:** Un valore 'estremo' non è necessariamente un errore di misurazione, ma una prova della complessità del mondo reale.
 - **Valida le relazioni tra variabili:** Conferma che il rapporto tra i dati (peso/altezza) può restare normale anche in presenza di valori assoluti fuori norma.

In sintesi, trattare l'outlier come un'opportunità invece che come un errore permette di non perdere informazioni cliniche o statistiche di vitale importanza.

- **Trasformazione dei dati:** è possibile applicare trasformazioni statistiche come alternativa all'esclusione. Ad esempio, si può applicare una trasformazione sotto radice o logaritmica per ridurre l'impatto dei valori anomali. Questo metodo è particolarmente utile quando gli outlier derivano da distribuzioni asimmetriche. Un approccio alternativo è la windsorization, che consiste nel sostituire i valori estremi con altri meno estremi.

- **Utilizzo di metodi statistici robusti:** per mitigare l'influenza degli outlier senza escluderli, si può ricorrere all'utilizzo di metodi statistici che sono meno sensibili agli outlier, come test che considerano la mediana al posto della media. Ciò offre un buon compromesso tra l'inclusione dei dati e la riduzione dell'impatto delle osservazioni anomale.

Connessione con il Machine Learning : *l'uso di metodi statistici robusti è fondamentale per creare modelli che non vengano "trascinati" da singoli dati anomali. Ecco come si applica:*

- Funzioni di Perdita (Loss Functions): Invece di usare l'Errore Quadratico Medio (MSE), che penalizza pesantemente gli outlier (essendo basato sulla media), si può usare l'Errore Assoluto Medio (MAE). Il MAE è più robusto perché si basa sulla mediana.
- Algoritmi basati su Alberi: Modelli come i Random Forest o il Gradient Boosting sono intrinsecamente più resistenti agli outlier rispetto alla regressione lineare, poiché dividono i dati in base a soglie e non tentano di adattare una curva globale.
- Regressione Robusta: Esistono varianti come la Huber Loss, che combina i vantaggi di media e mediana per gestire i dati estremi senza perdere efficacia.

*In ambito clinico è fondamentale **mantenere un elevato livello di trasparenza nel processo di valutazione e trattamento degli outlier**. Le decisioni sulla loro gestione devono essere chiaramente documentate e giustificate nei rapporti di ricerca, **valutandone l'impatto sui risultati finali**. L'esclusione di valori ritenuti "anomali" potrebbe portare a ignorare importanti sottogruppi di popolazione. Il trattamento degli outlier negli studi clinici è una questione delicata che richiede un equilibrio tra rigore statistico e considerazioni cliniche. Mentre gli outlier possono distorcere i risultati, essi possono anche **rappresentare informazioni cliniche di fondamentale importanza**.*

E' fondamentale documentare e giustificare ogni decisione presa riguardo alla loro gestione per garantire la robustezza e la validità dei dati ottenuti, assicurando che i risultati degli studi siano rappresentativi e affidabili.

Ecco una **checklist schematica** per la gestione degli outlier, basata sulle fonti e sull'analisi precedente:

1. **Identificazione e Origine**

- Metodo di rilevazione: Sono stati utilizzati box-plot, analisi dei residui o test statistici (Grubbs/Dixon)?

- Natura del dato: Si tratta di un errore di misurazione/registrazione o di una variazione biologica reale?
- Verifica coerenza: Il valore è coerente se rapportato ad altre variabili (es. peso elevato rapportato a statura molto alta)?

2. Valutazione dell'Impatto

- Distorsione statistica: Quanto vengono alterate la media e la deviazione standard?
- Validità inferenziale: L'outlier sta amplificando o attenuando l'efficacia apparente di un trattamento? → spiegare meglio
- Rilevanza clinica/settoriale: Il dato rappresenta un sottogruppo di popolazione importante che merita indagini separate?

3. Strategia di Trattamento

- Esclusione (Trimming): L'eliminazione è giustificata da un errore certo e opportunamente documentata?
- Trasformazione: È possibile usare logaritmi o radici per gestire distribuzioni asimmetriche?
- Sostituzione (Windsorization): È opportuno sostituire i valori estremi con valori meno distanti?
- Metodi robusti: Si può utilizzare la mediana al posto della media per ridurre la sensibilità alle anomalie? → generalizza è possibile usare metodi più robusti rispetto agli outlier

4. Integrazione nel Machine Learning

- Funzione di perdita: È preferibile l'uso del MAE (Errore Assoluto Medio) o della Huber Loss rispetto al MSE per limitare l'impatto degli outlier? (Informazione non presente nelle fonti, verificare indipendentemente).
- Scelta dell'algoritmo: Sono stati valutati modelli resistenti come i Random Forest o il Gradient Boosting? (Informazione non presente nelle fonti, verificare indipendentemente).

5. Trasparenza e Documentazione

- Giustificazione: Ogni scelta di mantenimento o rimozione è stata chiaramente motivata nel rapporto di ricerca?
- Bilanciamento: È stato raggiunto un equilibrio tra rigore statistico e valore clinico/di settore dell'informazione?

Standardizzazione delle stringe = testo = "ITALIA"

Descrizione, valutare se alcune variabili categoriche espresse in numeri è conveniente rasformarle in stringhe(mapping funzione vs repce)

- Il processo di **pulizia e standardizzazione dei dati** ha l'obiettivo fondamentale di trasformare le informazioni grezze, spesso frammentate o eterogenee, in una **forma coerente e ben definita**. Nelle basi di dati reali, le informazioni personali possono essere registrate in formati diversi, presentare errori di battitura, variazioni ortografiche o risultare obsolete. La standardizzazione agisce come un filtro che normalizza queste discrepanze, permettendo analisi successive e processi di collegamento (linkage) molto più accurati.
- Ambiti e Situazioni d'Uso

La standardizzazione delle stringhe risulta indispensabile in diverse situazioni critiche:

- **Record Linkage (Collegamento dei record)**: È essenziale per confrontare record provenienti da sorgenti differenti che non condividono un identificativo univoco. Senza una forma standard, è difficile stabilire se due stringhe diverse si riferiscano alla stessa entità.
- **Normalizzazione di Attributi Vari**: Si applica a diverse classi di dati, tra cui:
 - **Nomi e Indirizzi**: Spesso le componenti più soggette a variazioni.
 - **Date e Orari**: Necessitano di formati uniformi per essere elaborati statisticamente.
 - **Attributi Categorici**: Come il sesso o il paese di nascita, che devono essere ricondotti a valori standard precodificati.
- Accorgimenti per Migliorare il Processo

Per rendere il processo di standardizzazione realmente efficace, è consigliabile adottare i seguenti accorgimenti tecnici:

- a. **Segmentazione (Parsing)**: Invece di confrontare intere stringhe (come un intero indirizzo), è molto più efficace suddividere il dato in numerosi **campi di output più piccoli** (es. separare numero civico, nome della via, CAP). Questo permette un confronto granulare e aumenta drasticamente la qualità dei risultati.
- b. **Visualizzazione della distribuzione delle stringhe**: L'impiego di funzioni e tabelle per valutare la frequenza delle stringhe presenti in una colonna, possono essere usati anche grafici come bar plots. Questo aiuta a identificare possibili errori, identificare outliers, e quindi direzionare la correzione sistematica.
- c. **Approcci Statistici e Modelli Avanzati**: Per gestire l'ambiguità delle stringhe, si può passare da semplici sistemi basati su regole a metodi più sofisticati, come i **Modelli di Markov Nascosti (HMM)**, che sono particolarmente efficaci nella standardizzazione di nomi e indirizzi complessi.
- d. **Struttura a Fasi**: Un processo robusto dovrebbe sempre seguire una sequenza logica: Conosce le stringhe e categorie, identificare errori o outliers, uniformazione

delle stringhe secondo standard definito nel progetto, prestare particolare attenzione alle stringhe che indicano dati categorici.

- `testo.lower()` # "italia"
- `testo.upper()` # "ITALIA"
- `testo.capitalize()` # "Italia"
- `testo.title()` # "Italia"
- `testo.swapcase()` # "italia"
- `df.strip` → rimuove gli spazi all'inizio o alla fine
- `df["colonna"] = df["colonna"].str.replace(" ", "")` → rimuove TUTTI gli spazi
- `df["colonna"] = df["colonna"].str.replace("€", "")`
`df["colonna"] = df["colonna"].str.replace(",", "")` → rimuove simboli indesiderati
- `df["colonna"] = df["colonna"].apply(lambda x: x.lower()).str` → trasformazioni che implicano più di un processo

Convertire il tipo di dati dove necessario

La **conversione dei dati** serve a trasformare le informazioni originali in una **forma coerente e ben definita** per l'analisi. Utilizzando il linguaggio Python, è possibile scrivere funzioni specifiche per cambiare i tipi di dato (ad esempio, trasformando stringhe o tuple in numeri interi), assicurando che ogni valore sia nel formato corretto per i calcoli statistici o il collegamento dei record.

- `astype()` → **ES**: `df["colonna"] = df["colonna"].astype(int o float o str)`
- `pd.to_numeric()` → più specifico per dati numerici, trasforma i valori errati in NaN
- `pd.to_datetime()`

Particolare attenzione va data al significato dei dati oltre al loro tipo, ovvero un numero intero potrebbe non essere un dato quantitativo ma qualitativo. In questo caso, se si tratta di dati qualitativi nominali allora può essere trasformato in stringa ricodificandolo secondo il nome della classe (1: appartamento, 2: villetta, 3: garages, 4: non abitabile,...). Da valutare anche se utile per dati qualitativi ordinali.

3. Analisi dettagliata singolo file, conservativo

Ora che il dataset è ordinato e più leggibile andiamo a considerare la valenza del dato stesso, gli outliers, se tutti i dati sono necessari, se occorre armonizzarli.

Prendiamo in considerazione che dati (colonne) ci sono davvero necessari per le analisi da effettuare.

- Heatedmap → questa mappa ci permette di visualizzare più chiaramente gli elementi di correlazione tra variabili
- `df.corr()` → viene utilizzato per calcolare la **correlazione tra le colonne numeriche** di un DataFrame. Questa funzione permette di determinare quanto sia forte la relazione tra due diverse variabili. (funziona solo su variabili numeriche)
 - **Metodi speciali (Parametri)**

La funzione permette di scegliere diversi algoritmi per il calcolo attraverso il parametro `method`:

- **Pearson**: È il metodo predefinito. Utilizza la covarianza dei valori effettivi per trovare relazioni lineari.
- **Spearman**: Un metodo basato sulla differenza dei ranghi (posizioni in classifica), utile per identificare relazioni perfette anche se non lineari.
- **Kendall**: Calcola la correlazione basandosi sulle coppie di ranghi concordanti e discordanti.

Oltre al metodo, si possono specificare parametri come `min_periods` (numero minimo di osservazioni valide richieste) e `numeric_only` (se includere solo colonne numeriche) da riscrivere meglio

Questi non sono gli unici metodi disponibili. E' possibile specificare alcuni parametri su cui si voglia lavorare con altri contenuti di calcolo(es: `min_periods`, `numeric_only`)

- **Visualizzazione e lettura dell'output**

Il risultato della funzione è un nuovo DataFrame chiamato **matrice di correlazione**, dove ogni valore rappresenta il legame tra due colonne. I dati in output vanno letti su una scala che va da **-1 a 1**:

- **1**: Indica una **correlazione positiva perfetta**. Se una colonna aumenta, l'altra aumenta in modo perfettamente proporzionale.
- **-1**: Indica una **correlazione negativa perfetta**. Se una colonna aumenta, l'altra diminuisce in modo proporzionale.
- **0**: Indica l'**assenza di correlazione** tra le variabili.
- **Valori intermedi**: Valori vicini a 1 (come 0.97) indicano una forte relazione positiva (es. altezza e peso), mentre valori intorno a -0.6 indicano una relazione negativa moderata (es. ore di studio e livello di stress).

Domande da porsi per **valutare la valenza** di una colonna

Per aiutarti operativamente a fare questa selezione sul tuo dataset, puoi sottoporre ogni colonna a questo test logico:

- **Pertinenza con l'obiettivo**: Questa colonna aiuta direttamente a rispondere alla domanda della mia analisi? (Es: se analizzi il churn dei clienti, l'altezza del cliente non serve, il fatturato sì).
- **Valore informativo unico**: Questa informazione è già espressa (meglio) in un'altra colonna? (Es: se hai sia "Data di nascita" che "Età", una delle due è ridondante).
- **Qualità del dato di fondo**: Anche se la colonna è importante, i dati al suo interno sono abbastanza affidabili o sono pieni di errori di inserimento manuale che falserebbero l'analisi?
- **Variabilità**: Il dato varia abbastanza da mostrare un trend, o è quasi identico per tutti i record?
- **Unione**: è possibile riassumere due variabili in un'unica? Ad esempio se ci sono molteplici variabili di qualità per le medesime variabili, posso riassumerle in un'unica variabile, documentare precisamente la decisione per poterla replicare in altri file.

4. Armonizzazione variabili utili (in base agli algoritmi)

unità di misura, il tipo, valori categorici rappresentazione → verifica senza cancellare dei due file.

se ci sono delle variabili comuni tra i dataset ma con origine di misurazione differente e non armonizzabile in maniera semplice allora indicare il metodo di ottenimento in un'altra colonna.

Cos'è l'Armonizzazione dei Dati?

L'armonizzazione è la pratica di **conciliare dati provenienti da diversi tipi, livelli e fonti** per renderli compatibili e comparabili, facilitando così analisi e processi decisionali migliori. Si distingue dall'integrazione dei dati (che unisce set concettualmente diversi) e dalla standardizzazione (che impone una singola metodologia).

Per creare un dataset armonizzato, è necessario risolvere l'eterogeneità lungo tre dimensioni:

- **Sintassi**: Formati tecnici differenti (es. .csv, JSON, HTML).
- **Struttura**: Come le variabili si relazionano tra loro (es. dati strutturati in tabelle vs dati non strutturati).
- **Semantica**: Il significato inteso delle parole e delle misure (es. diverse definizioni di "giovani adulti" in base all'età).

Tipologie e Implementazione

L'armonizzazione può essere approcciata in diversi modi:

- **Prospettica vs Retrospettiva:** L'armonizzazione **prospettica** avviene prima della raccolta dei dati tramite il coordinamento delle metodologie, mentre quella **retrospettiva** agisce su dataset già esistenti.
- **Stringente vs Flessibile:** La prima utilizza procedure identiche tra gli studi; la seconda punta a rendere i dati "inferenzialmente equivalenti" anche se non identici.
- **Merging vs Mapping:** Il **merging** crea una nuova tassonomia globale che include tutte le informazioni, mentre il **mapping** stabilisce regole per collegare tassonomie diverse tra loro.

5. Unione file → pulizia file unito → tenere il necessario

A questo punto diamo per scontato di aver già intrapreso un'analisi dei dataset che intendiamo unire singolarmente. In questo modo non ci sarà bisogno di una pulizia approfondita ma bensì un'armonizzazione tra le due o più che siano.

- Focalizzare sulla pulizia finale post unione
- Puoi fare in cattivo con la rimozione
- rimozione colonne che mi sono servite per la pulizia precedentemente, generica
- togliere il superfluo

Una volta completata l'analisi e la pulizia dei singoli dataset, il focus si sposta dall'integrazione alla vera e propria **armonizzazione retrospettiva**. In questa fase, l'obiettivo non è più correggere il singolo errore di battitura, ma risolvere le eterogeneità di **sintassi, struttura e semantica** che emergono solo dopo l'unione delle fonti.

Ecco come gestire la **pulizia finale e la rimozione del superfluo** per ottenere un dataset armonizzato e pronto per l'analisi:

1. Eliminazione delle "Colonne di Servizio"

Durante il processo di pulizia iniziale e standardizzazione, è comune suddividere i dati originali in numerosi **campi di output più piccoli** (segmentazione) per facilitare il collegamento dei record (es. separare un indirizzo in via, civico e CAP).

- **Azione:** Una volta che l'unione (merging) è andata a buon fine e i record sono stati collegati correttamente, queste colonne tecniche di segmentazione e i tag temporanei utilizzati dalle tabelle di consultazione diventano superflui.
- **Argomentazione:** Mantenere queste colonne "di passaggio" appesantisce il dataset, rendendolo **"information-poor" (superficiale)**, ovvero con molti campi che non aggiungono valore statistico ma aumentano solo la complessità computazionale.

2. Rimozione aggressiva della ridondanza

L'unione di più dataset porta spesso con sé una quota di **ridondanza strutturale**.

- **Deduplicazione post-unione:** Una volta combinati i dataset, è imperativo procedere alla **deduplicazione**. Se due fonti riportano lo stesso evento clinico o demografico, mantenere entrambi i record creerebbe un bias artificiale nei risultati finali.
- **Scelta della tassonomia dominante:** Se hai utilizzato il metodo del *mapping* (creare regole per collegare ontologie diverse), potresti ritrovarti con variabili che misurano lo stesso concetto in modi leggermente diversi. In questo caso, bisogna "fare i cattivi": scegli la variabile con la **granularità più adatta** e rimuovi le altre varianti che non verranno utilizzate nell'analisi.

3. Finalizzazione del "DataSchema"

La pulizia finale serve a definire il **DataSchema** definitivo, ovvero l'insieme delle variabili chiave selezionate per la ricerca.

- **Togliere il superfluo:** Ogni colonna che non rientra nel DataSchema o che non serve a validare la relazione tra le variabili (come nell'esempio del rapporto peso/altezza) deve essere eliminata.
- **Perché farlo:** Un dataset pulito e privo di variabili inutilizzate facilita la **trasparenza** e la successiva documentazione dei risultati, permettendo ad altri ricercatori di valutare correttamente la qualità dell'armonizzazione effettuata.

In sintesi, la pulizia post-unione è un processo di **rifinitura** dove la rimozione degli elementi superflui non è solo un atto tecnico, ma una scelta metodologica per garantire che il dataset finale rifletta esclusivamente le informazioni necessarie per rispondere ai quesiti della ricerca.

6. Normalizzazione

tipi di normalizzazione e caratteristiche

Introduzione alla Normalizzazione dei Dati

La **normalizzazione dei dati** è un processo di trasformazione delle caratteristiche numeriche volto a garantire che esse occupino intervalli comparabili o seguano una scala coerente. In molti contesti di analisi e machine learning, le variabili di input presentano spesso scale molto diverse tra loro; senza un adeguato intervento di preprocessing, gli algoritmi potrebbero attribuire un'importanza sproporzionata alle variabili con valori numericamente più elevati, distorcendo i risultati del modello.

Obiettivi e Rilevanza Clinica e Statistica

L'adozione di tecniche di normalizzazione risponde a diverse necessità metodologiche:

- **Ottimizzazione dei Modelli:** Algoritmi basati sulla discesa del gradiente (come le reti neurali) o sulle distanze (come kNN) richiedono scale uniformi per evitare instabilità

nell'addestramento o bias nelle previsioni.

- **Confronto Equo tra Variabili:** Permette di combinare misurazioni espresse in unità differenti (ad esempio, età in decenni e reddito in migliaia) su un piano di parità, riflettendo il loro reale valore predittivo anziché la loro magnitudo accidentale.
- **Integrità e Qualità del Dato:** La normalizzazione facilita l'integrazione di dati provenienti da fonti eterogenee con convenzioni inconsistenti, migliorando la coerenza complessiva del dataset.

Panoramica delle Tecniche

La scelta della strategia di normalizzazione dipende strettamente dalla distribuzione dei dati e dalla presenza di valori anomali:

1. **Min-Max Normalization:** Comprime i dati in un intervallo fisso (solitamente 0-1), preservando la forma della distribuzione originale; è ideale per dati con limiti naturali noti ma sensibile agli outlier.

$$x_{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

- **A cosa prestare attenzione:**

Gli outlier possono rovinare completamente la normalizzazione min-max. Immagina che alcuni dati si trovino principalmente tra 10 e 100, ma un valore sia 99.999? Quel singolo punto allunga così tanto la tua gamma che tutto ciò che è normale si comprime in una piccola fessura quasi zero. Una brutta osservazione, intera funzione rovinata.

Considera anche cosa succede quando nuovi dati escono dal raggio di addestramento. I valori di test oltre il massimo di addestramento producono output superiore a 1. Se questo rompa qualcosa dipende dal tuo modello.

1. **Z-score Standardization:** Centra i dati sulla media con una deviazione standard unitaria. È considerata una scelta solida quando non si conoscono i limiti dei dati o per algoritmi che assumono distribuzioni normali.

$$z = \frac{x - \mu}{\sigma}$$

- **Compromesso:**

L'interpretabilità ne subisce un colpo. "Questa osservazione è 1,7 deviazioni standard sopra la media" richiede più lavoro mentale di "questo è all'80% dell'intervallo." Se gli stakeholder devono comprendere direttamente i valori scalati, il min-max potrebbe comunicare meglio.

Gli outlier gonfiano ancora la media e lo sviluppo standard, distorcendo la scalatura per i dati in massa, ma sono meno problematici rispetto alla normalizzazione min-max.

1. **Robust Scaling:** Utilizzando la mediana e l'intervallo interquartile, questa tecnica risulta la più efficace per gestire dataset "sporchi" o con outlier significativi che potrebbero distorcere le statistiche basate sulla media.

$$x_{robust} = \frac{x - median}{IQR}$$

- **Perché aiuta con dati distorti:**

Le distribuzioni del reddito sono un buon esempio. La maggior parte delle persone guadagna cifre moderate, e alcune guadagnano cifre enormi: una classica distribuzione a destra. La deviazione standard viene gonfiata dai lavoratori con reddito più alto, il che rende la standardizzazione meno utile per la maggior parte delle osservazioni. La mediana e l'IQR rimangono ancorate al centro della distribuzione comunque.

Best Practice Metodologiche

Per garantire la validità scientifica dell'analisi, è fondamentale che il processo di normalizzazione avvenga **dopo la suddivisione tra set di addestramento e set di test**. Adattare (fitting) il normalizzatore sull'intero dataset comporterebbe un fenomeno di *data leakage*, in cui informazioni del set di test influenzano il modello, invalidando l'accuratezza delle metriche di valutazione finali.

La fonte fornita approfondisce il tema della **normalizzazione dei dati** utilizzando la libreria Python **Scikit-Learn**, definendola un passaggio cruciale per migliorare le prestazioni e la stabilità dei modelli di machine learning.

Ecco una sintesi dei punti principali trattati:

Perché la normalizzazione è necessaria?

Molti algoritmi di machine learning sono sensibili alla scala delle caratteristiche (features). Se una variabile ha un range molto più ampio delle altre, può dominare il modello portando a risultati imprecisi. La normalizzazione assicura che ogni caratteristica contribuisca equamente al modello. È particolarmente importante per algoritmi basati sul calcolo della distanza (come **KNN** o **SVM**) e per quelli che utilizzano la **discesa del gradiente**.

Tecniche principali in Scikit-Learn

L'articolo descrive tre trasformatori fondamentali:

- **Min-Max Scaling (MinMaxScaler):** Riscalda i dati all'interno di un intervallo specifico, solitamente tra **0 e 1**. È utile quando i range delle variabili variano significativamente e si desidera preservare un intervallo preciso.
- **Standardizzazione / Z-Score (StandardScaler):** Trasforma le caratteristiche affinché abbiano una **media di 0 e una deviazione standard di 1**. È indicata quando la distribuzione dei dati non è uniforme e si vogliono preservare le proprietà statistiche di base.
- **Robust Scaling (RobustScaler):** Questa tecnica utilizza la **mediana** e il **range interquartile (IQR)** per scalare i dati. La sua caratteristica principale è la **robustezza agli outlier**, poiché queste statistiche sono meno influenzate dai valori anomali rispetto alla media e alla deviazione standard.

Buone pratiche e considerazioni

- **Gestione degli Outlier:** I valori anomali possono influenzare pesantemente la normalizzazione; in questi casi, è preferibile usare il **Robust Scaling** o gestire gli outlier prima della trasformazione.
- **Reversibilità:** Le trasformazioni effettuate con Scikit-Learn possono essere invertite utilizzando il metodo `inverse_transform`.
- **Combinazione di tecniche:** In scenari complessi, è possibile applicare tecniche diverse a caratteristiche diverse all'interno dello stesso dataset.

In conclusione, la scelta del metodo corretto dipende dal contesto e dalla natura dei dati, ma integrare la normalizzazione nel processo di preprocessing garantisce previsioni più accurate e affidabili.

Ora che il dataset è ordinato e più leggibile andiamo a considerare la valenza del dato stesso, gli outliers, se tutti i dati sono necessari, se occorre armonizzarli.

N.B. : Se stiamo unendo due dataset è molto importante capire se i dati sono allineati fra loro, ossia se hanno le stesse metriche, armonizzate e primary key che ci permette di intersecare i dati.

- Aggiungere eventuali codici utili per la pulizia
- Normalizzazione dati(massimo, minimo, dev, media) Fare dai range
- Più approfondita sul singolo file prima dell'unione tra loro
- Armonizzazione alla fine di tutto